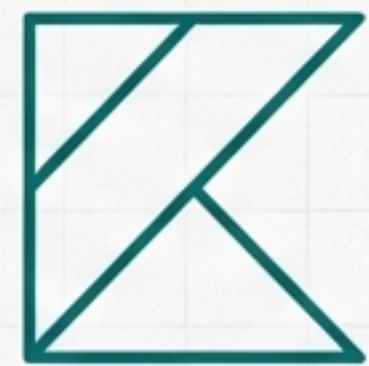


De la Idea al Código: Construyendo un Marketplace Moderno

Un análisis arquitectónico de una aplicación Android desarrollada con Programación Dirigida por Eventos y el patrón MVVM.



Un Marketplace Robusto Construido Sobre Fundamentos Modernos

Este proyecto es una aplicación de marketplace para Android, creada para la asignatura "Programación Dirigida por Eventos". El objetivo es demostrar la aplicación de conceptos de desarrollo moderno para construir una aplicación eficiente, escalable y centrada en el usuario.



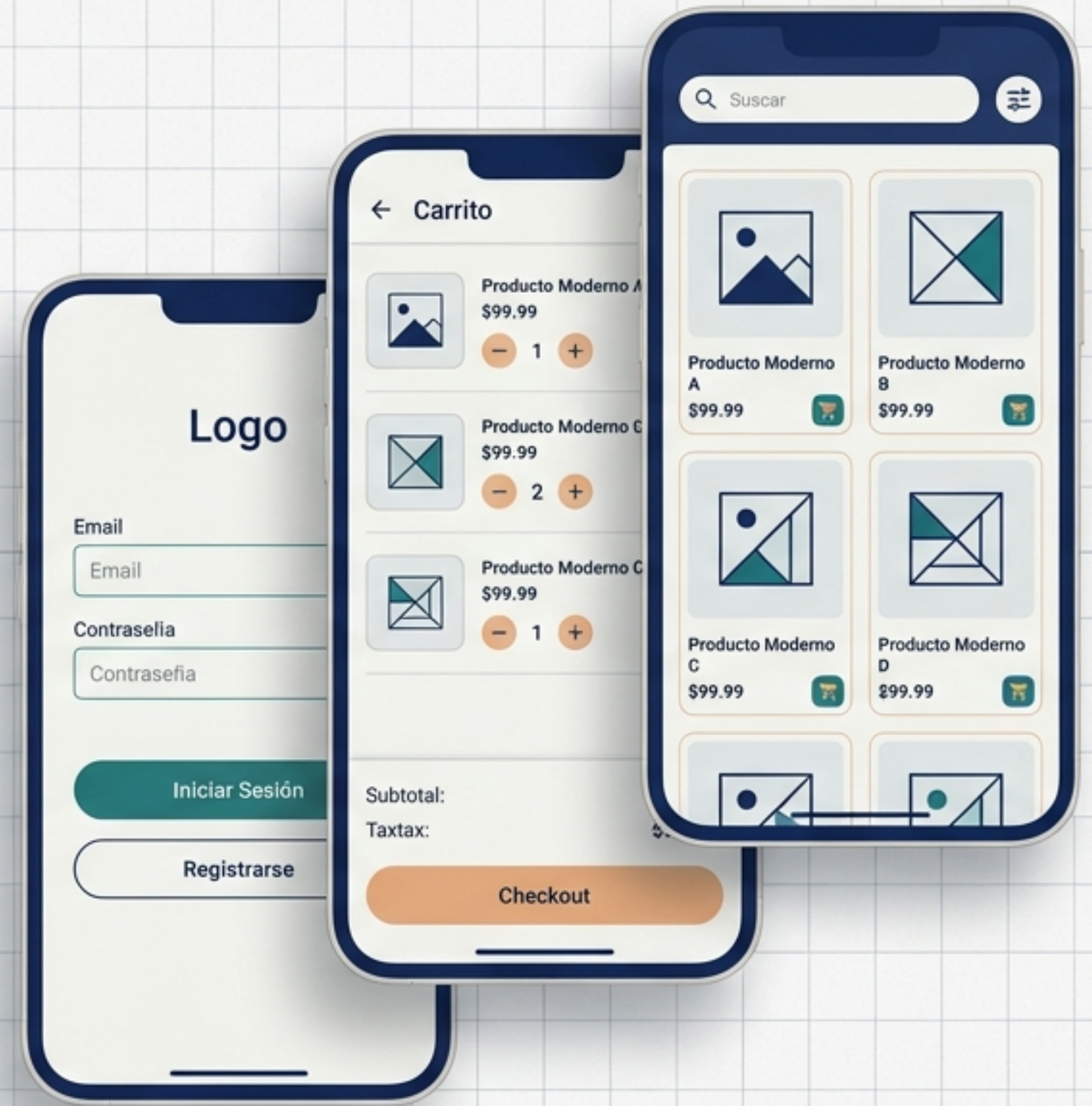
Autenticación de Usuarios: Registro e inicio de sesión seguros.



Catálogo de Productos: Exploración fluida de artículos disponibles.



Carrito de Compras: Gestión de selecciones y persistencia de datos.



El Plano Arquitectónico: Una Sola Actividad con el Patrón MVVM

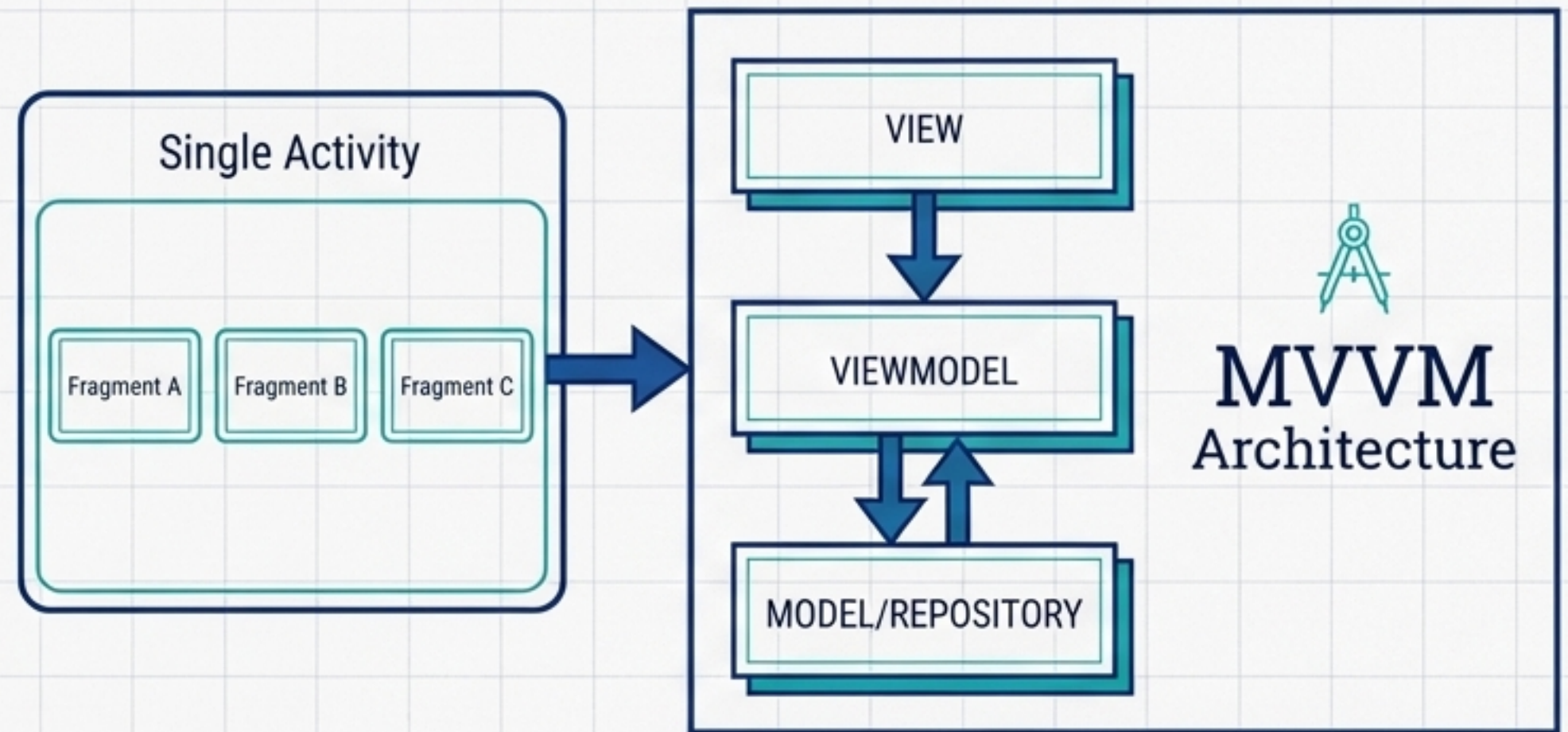
Arquitectura de Actividad Única (Single-Activity)

Se utiliza una única **`Activity`** que actúa como contenedor para múltiples **`Fragments`**. Esto simplifica la gestión del ciclo de vida y la navegación.

Patrón de Diseño MVVM (Model-View-ViewModel)

Una arquitectura que separa la lógica de la interfaz (Vistas) de la lógica de negocio (ViewModels) y los datos (Repositorios).

- * Beneficios: Código más organizado, alta escalabilidad y facilidad para realizar pruebas unitarias.



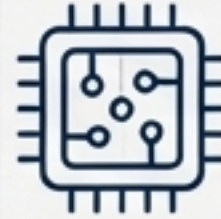
Las Capas de Nuestra Construcción: Descomponiendo MVVM



Vistas (Views - Fragments & Activity)

Responsabilidad: Mostrar los datos y capturar los eventos del usuario (clics, gestos).

- **Regla de Oro:** No contienen ninguna lógica de negocio.



ViewModel

Responsabilidad: Almacenar y gestionar los datos relacionados con la UI, exponiéndolos a través de **LiveData**.

- **Característica Clave:** Sobrevive a cambios de configuración (ej. rotación de pantalla), evitando la pérdida de estado.



Repositorio (Repository)

Responsabilidad: Actúa como la única fuente de verdad (*Single Source of Truth*). Abstrae el origen de los datos (base de datos local, servicios web) del resto de la app.

La Interacción es Clave: Programación Dirigida por Eventos

La interactividad de la aplicación se basa en la respuesta a eventos generados por el usuario.



1. ****Acción del Usuario***

Un usuario hace clic un botón (ej. 'Añadir al carrito').

2. ****Disparador (Listener)***

El ``setOnClickListener`` captura el evento.

3. ****Procesamiento**

El evento desencadena una función en el ``ViewModel`` para actualizar el estado de la aplicación.

4. ****Actualización de UI**

LiveData notifica a la Vista del cambio de estado y la interfaz se actualiza automáticamente.

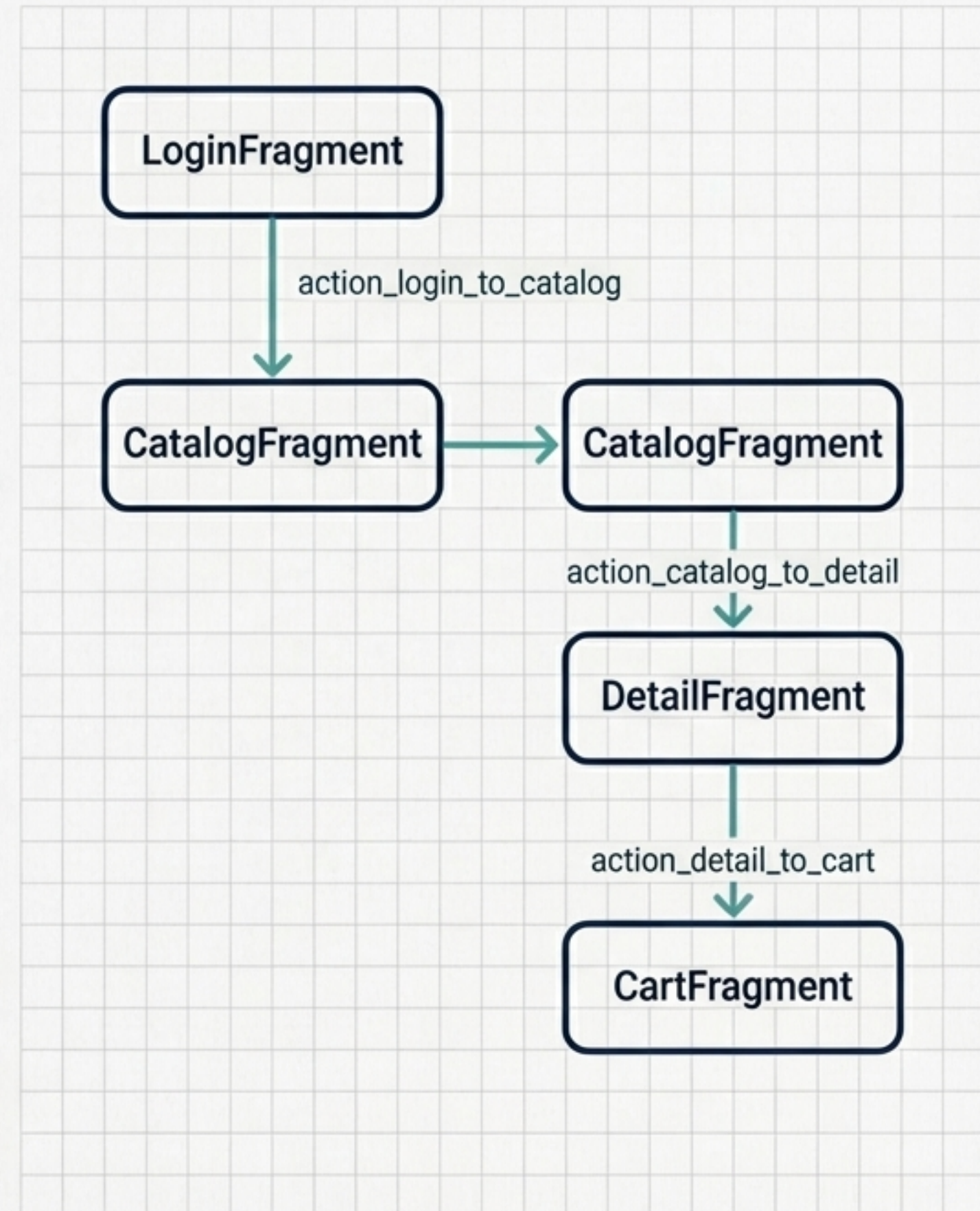
Un Flujo de Usuario Claro y Centralizado con Navigation Component

El Problema que Resuelve: Gestionar la navegación entre múltiples fragmentos puede volverse complejo y propenso a errores.

Nuestra Solución: Usamos **Navigation Component**, una librería de Android Jetpack que centraliza todo el flujo de navegación.

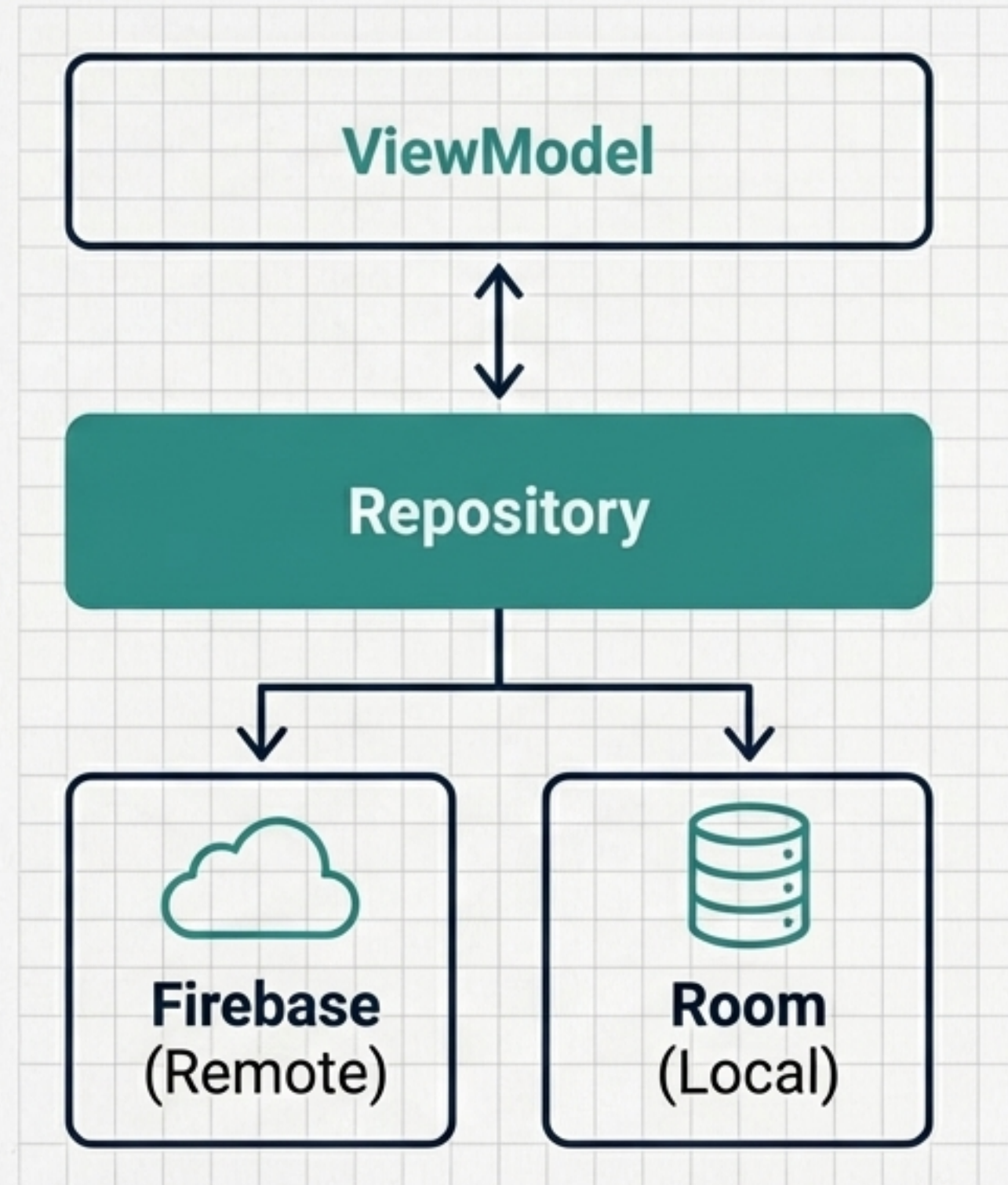
How it Works

- Toda la navegación se define visualmente en un único archivo: `navigation/nav\_graph.xml`.
- Se definen 'acciones' que conectan un destino (fragmento) con otro.
- Maneja automáticamente la pila de retroceso (*back stack*), asegurando un comportamiento predecible para el usuario.



La Única Fuente de Verdad: El Patrón Repositorio

Principio Fundamental:
El resto de la aplicación (especialmente los ViewModels) no necesita saber de dónde vienen los datos, solo cómo solicitarlos.



Rol del Repositorio:

- Abstrae las fuentes de datos.
- Decide si obtener datos de una API o de la base de datos local.
- Centraliza toda la lógica de acceso a datos.

Asegurando el Acceso: Autenticación con Firebase

Tecnología Utilizada: **Firebase Authentication** como servicio de backend.

Registro de Nuevos Usuarios: Creación de cuentas con correo electrónico y contraseña.

Inicio de Sesión: Verificación de credenciales de usuarios existentes.

Ventaja Estratégica: Permite una implementación rápida y segura de un sistema de autenticación robusto sin necesidad de gestionar un servidor propio.



Perfiles de
Usuario



Seguridad



Acceso por
Email

Persistencia Local de Datos con Room

Tecnología Utilizada:

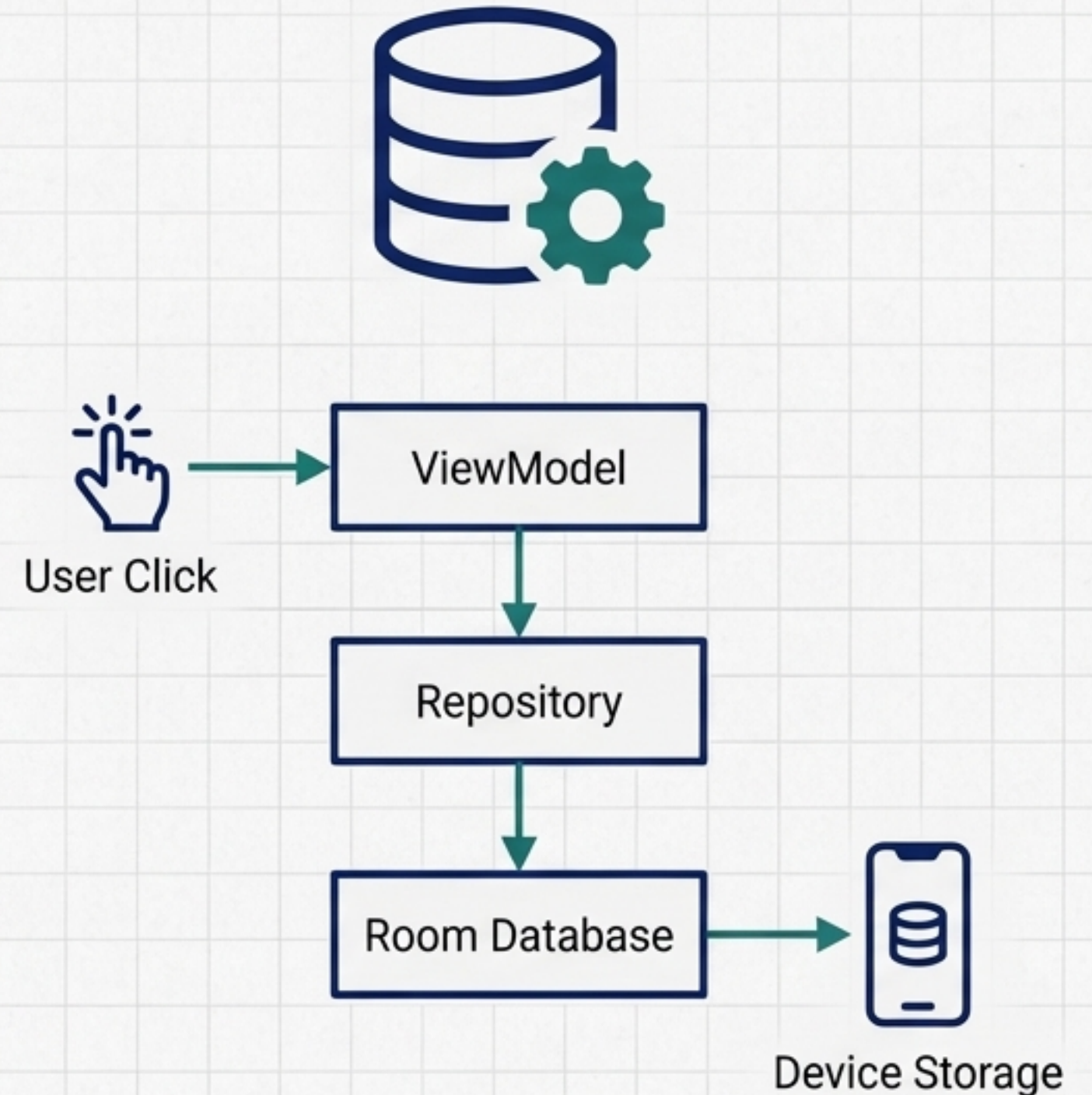
La biblioteca **Room**, una capa de abstracción sobre SQLite.

Casos de Uso en el Proyecto:

- * Almacenamiento del **carrito de compras** del usuario.
- * Persistencia de un futuro **historial de pedidos**.

Ventaja Estratégica:

Room reduce el código repetitivo (*boilerplate*), verifica las consultas SQL en tiempo de compilación y se integra perfectamente con **LiveData** y corrutinas.

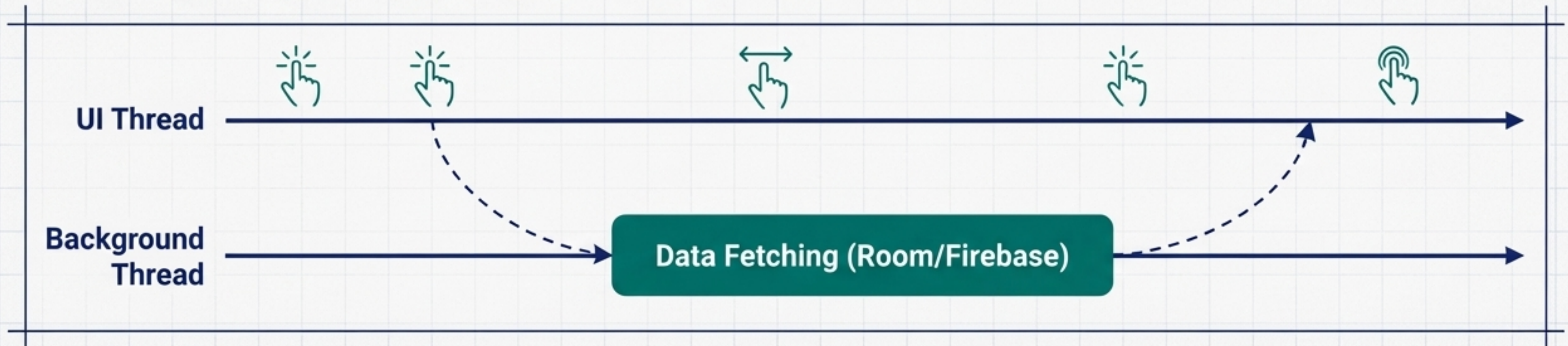


Una UI Fluida: Gestión de Tareas Asíncronas con Corrutinas

El Desafío: Las operaciones de red (Firebase) y de acceso a disco (Room) son lentas y no pueden ejecutarse en el hilo principal (*main thread*) sin bloquear la interfaz de usuario.

La Solución: Utilizamos **Corrutinas de Kotlin** para ejecutar estas operaciones de forma asíncrona.

Las funciones en los `Repositorios` y `ViewModels` que acceden a datos son marcadas como `suspend`, asegurando que la UI permanezca siempre receptiva.



Más Allá de la Aplicación: Potencial para un Impacto Sostenible

La base tecnológica de este marketplace es flexible y puede adaptarse para abordar Objetivos de Desarrollo Sostenible (ODS).



ODS 8 (Trabajo Decente y Crecimiento Económico)

La plataforma podría servir para que pequeños productores y artesanos locales vendan sus productos, promoviendo el comercio justo y la economía local.



ODS 12 (Producción y Consumo Responsables)

La app podría especializarse en la venta de productos de segunda mano, artículos reciclados o productos de origen sostenible, fomentando una economía circular.

Manual de Construcción: Cómo Ejecutar el Proyecto

1



Clonar el repositorio desde la fuente.

2



Abrir en Android Studio usando la opción 'Open an existing project'.

3



Configurar Firebase

- Crear un proyecto en la Consola de Firebase.
- Añadir una app Android con el paquete `com.example.marketplace`.
- Descargar el archivo `google-services.json` y copiarlo en la carpeta `app/`.
- Habilitar el proveedor de autenticación 'Correo electrónico/Contraseña'.

4



Construir y Ejecutar

- Sincronizar Gradle y pulsar 'Run' en un emulador o dispositivo.

El Blueprint Completo: Una Arquitectura Cohesiva y Moderna

Este diagrama encapsula la interacción de todos los componentes clave discutidos, demostrando un flujo de datos y eventos limpio y desacoplado.

